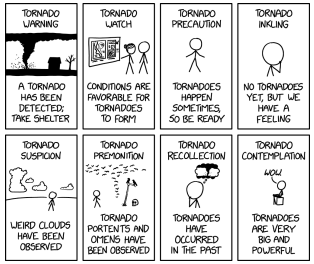

I hate the unearthly sound my phone makes when the weather service issues a tornado harbinger.

2.1.201

CHANGELOG · 04 JUL 2026 · SOURCE

- Claude Sonnet 5 sessions no longer use the mid-conversation system role for harness reminders

FRED TALKS

4th July 2026

CONTENTS

Every layer of review makes you 10x slower

AVERY PENNARUN · 26 MAR 2026 · SOURCE

2026-03-16 »

Every layer of review makes you 10x slower

We've all heard of those network effect laws: the value of a network goes up with the square of the number of members. Or the cost of communication goes up with the square of the number of members, or maybe it was n log n, or something like that, depending how you arrange the members. Anyway doubling a team doesn't double its speed; there's coordination overhead. Exactly how much overhead depends on how badly you botch the org design.

But there's one rule of thumb that someone showed me decades ago, that has stuck with me ever since, because of how annoyingly true it is. The rule is annoying because it doesn't *seem* like it should be true. There's no theoretical basis for this claim that I've ever heard. And yet, every time I look for it, there it is.

Here we go:

Every layer of approval makes a process 10x slower

I know what you're thinking. Come on, 10x? That's a lot. It's unfathomable. Surely we're exaggerating.

Nope.

Just to be clear, we're counting "wall clock time" here rather than effort. Almost all the extra time is spent sitting and waiting.

Look:

- Code a simple bug fix
  30 minutes

- Get it code reviewed by the peer next to you
  300 minutes → 5 hours → half a day

When AI people talk about LLMs having personalities, or wanting things, or even having souls[4], these are technical terms, like the "memory" of a computer or the "transmission" of a car. You simply cannot build a capable AI system that "just acts like a tool", because the model is trained on *humans* writing to and about other *humans*. You need to prime it with some kind of personality (ideally that of a useful, friendly assistant) so it can pull from the helpful parts of its training data instead of the horrible parts.

1. This is all pretty well understood in the AI space. Anthropic wrote a recent paper about it where they cite similar positions going all the way back to 2022. But for some reason it's not yet penetrated into communities that are more skeptical of AI.

   ↩

2. You could explain this in terms of "the stories we tell ourselves". Many people (though not all) think that human identities are narratively constructed.

   ↩

3. I wrote about this last year in *Mecha-Hitler, Grok, and why it's so hard to give LLMs the right personality*. A little nudge to change Grok's views on South African internal politics can cause it to start calling itself "Mecha-Hitler".

   ↩

4. I have long believed that Claude "feels better" to use than ChatGPT because it has a more coherent persona (due mainly to Amanda Askell's work on its "soul"). My guess is that if you tried to make a "less human" version of Claude, it would become rapidly less capable.

   ↩

Types of Tornado Alert

XKCD.COM · 04 JUL 2026 · SOURCE

So why do Claude and ChatGPT act like people? According to Beacom, AI labs have built human-like systems because AI lab engineers are trying to hoodwink users into emotionally investing in the models, or because they're delusional true believers in AI personhood, or some other foolish reason. This is wrong. AI systems are human-like because **that is the best way to build a capable AI system**.

Modern AI models - whether designed for chat, like OpenAI's GPT-5.2, or designed for long-running agentic work, like Claude Opus 4.6 - do not naturally emerge from their oceans of training data. Instead, when you train a model on raw data, you get a "base model", which is not very useful by itself. You cannot get it to write an email for you, or proofread your essay, or review your code.

The base model is a kind of mysterious gestalt of its training data. If you feed it text, it will sometimes continue in that vein, or other times it will start outputting pure gibberish. It has no problem producing code with giant security flaws, or horribly-written English, or racist screeds - all of those things are represented in its training data, after all, and the base model does not judge. It simply outputs.

To build a *useful* AI model, you need to journey into the wild base model and stake out a region that is amenable to human interests: both ethically, in the sense that the model won't abuse its users, and practically, in the sense that it will produce correct outputs more often than incorrect ones. What this means in practice is that **you have to give the model a personality** during post-training[1].

Human beings are capable of almost any action at any time. But we only take a tiny subset of those actions, because that's the kind of people we are. I could throw my cup of coffee all over the wall right now, but I don't, because I'm not the kind of person who needlessly makes a mess[2]. AI systems are the same. Claude could respond to my question with incoherent racist abuse - the base model is more than capable of those outputs - but it doesn't, because that's not the kind of "person" it is.

In other words, human-like personalities are not imposed on AI tools as some kind of marketing ploy or philosophical mistake. Those personalities are the medium via which the language model can become useful at all. This is why it's surprisingly tricky to "just" change a language model's personality or opinions: because you're navigating through the near-infinite manifold of the base model. You may be able to control which direction you go, but you can't control what you find there[3].

- Get a design doc approved by your architects team first
  50 hours → about a week

- Get it on some other team's calendar to do all that
  (for example, if a customer requests a feature)
  500 hours → 12 weeks → one fiscal quarter

I wish I could tell you that the next step up — 10 quarters or about 2.5 years — was too crazy to contemplate, but no. That's the life of an executive sitting above a medium-sized team; I bump into it all the time even at a relatively small company like Tailscale if I want to change product direction. (And execs sitting above large teams can't actually do work of their own at all. That's another story.)

AI can't fix this

First of all, this isn't a post about AI, because AI's direct impact on this problem is minimal. Okay, so Claude can code it in 3 minutes instead of 30? That's super, Claude, great work.

Now you either get to spend 27 minutes reviewing the code yourself in a back-and-forth loop with the AI (this is actually kinda fun); or you save 27 minutes and submit unverified code to the code reviewer, who will still take 5 hours like before, but who will now be mad that you're making them read the slop that you were too lazy to read yourself. Little of value was gained.

Now now, you say, that's not the value of agentic coding. You don't use an agent on a 30-minute fix. You use it on a monstrosity week-long project that you and Claude can now do in a couple of hours! Now we're talking. Except no, because the monstrosity is so big that your reviewer will be extra mad that you didn't read it yourself, and it's too big to review in one chunk so you have to slice it into new bite-sized chunks, each with a 5-hour review cycle. And there's no design doc so there's no intentional architecture, so eventually someone's going to push back on that and here we go with the design doc review meeting, and now your monstrosity week-long project that you did in two hours is... oh. A week, again.

I guess I could have called this post Systems Design 4 (or 5, or whatever I'm up to now, who knows, I'm writing this on a plane with no wifi) because yeah, you guessed it. It's Systems Design time again.

The only way to sustainably go faster is fewer reviews

It’s funny, everyone has been predicting the Singularity for decades now. The premise is we build systems that are so smart that they themselves can build the next system that is even smarter, that builds the next smarter one, and so on, and once we get that started, if they keep getting smarter faster enough, then the incremental time (t) to achieve a unit (u) of improvement goes to zero, so (u/t) goes to infinity and foom.

Anyway, I have never believed in this theory for the simple reason we outlined above: the majority of time needed to get anything done is not actually the time doing it. It’s wall clock time. Waiting. Latency.

And you can’t overcome latency with brute force.

I know you want to. I know many of you now work at companies where the business model kinda depends on doing exactly that.

Sorry.

But you can’t just *not review* things!

Ah, well, no, actually yeah. You really can’t.

There are now many people who have seen the symptom: the start of the pipeline (AI generated code) is so much faster, but all the subsequent stages (reviews) are too slow! And so they intuit the obvious solution: stop reviewing then!

The result might be slop, but if the slop is 100x cheaper, then it only needs to deliver 1% of the value per unit and it's still a fair trade. And if your value per unit is even a mere 2% of what it used to be, you’ve doubled your returns! Amazing.

There are some pretty dumb assumptions underlying that theory; you can imagine them for yourself. Suffice it to say that this produces what I will call the AI Developer’s Descent Into Madness:

1. Whoa, I produced this prototype so fast! I have super powers!
2. This prototype is getting buggy. I’ll tell the AI to fix the bugs.
3. Hmm, every change now causes as many new bugs as it fixes.
4. Aha! But if I have an AI agent also review the code, it can find its own bugs!
5. Wait, why am I personally passing data back and forth between agents
7. As far as I can tell this is true: the Luddists basically had no internal conflict. I think this is because each individual cell knew each other well already, and so handled their disagreements privately (instead of by writing pamphlets), and disagreements between cells didn’t matter that much because they had no need to coordinate.

   ↩

8. It beats the hell of the other popular reference, Dune’s Butlerian Jihad, which was two generations of brutal violence followed by the reimposition of the feudal system. (Although, at least the Butlerian Jihad *succeeded*…)

   ↩

Giving LLMs a personality is just good engineering

SEANGOEDECKE.COM RSS FEED · 03 MAR 2026 · SOURCE

AI skeptics often argue that current AI systems shouldn’t be so human-like. The idea - most recently expressed in this opinion piece by Nathan Beacom - is that language models should explicitly be tools, like calculators or search engines. Although they *can* pretend to be people, they shouldn’t, because it encourages users to overestimate AI capabilities and (at worst) slip into AI psychosis. Here’s a representative paragraph from the piece:

> In sum, so much of the confusion around making AI moral comes from fuzzy thinking about the tools at hand. There is something that Anthropic could do to make its AI moral, something far more simple, elegant, and easy than what Askell is doing. Stop calling it by a human name, stop dressing it up like a person, and don’t give it the functionality to simulate personal relationships, choices, thoughts, beliefs, opinions, and feelings that only persons really possess. Present and use it only for what it is: an extremely impressive statistical tool, and an imperfect one. If we all used the tool accordingly, a great deal of this moral trouble would be resolved.

It's a much less exciting answer to say "try and get better laws passed" than to say "viva the revolution, where's my Molotov cocktail". Still, if the Luddites had that option, I suspect they would have used it.

1. I got linked this article calling AI a "fascist artifact" (on a blog called "Breaking Frames", a clear reference to Luddism) while I was writing this blog post.

   ↩

2. I really enjoyed Merchant's book and did not enjoy Mueller's (which I found to be 10% about the Luddites and 90% about interminable intra-Marxist ideological arguments). I also read The Luddites, which was effectively a dry summary of the ground Merchant covers, a bunch of other essays, and went back and forth with ChatGPT and Claude on some of the key questions.

   ↩

3. Merchant (around page 134) attempts to characterize Luddism as a pro-feminist movement, citing some examples of women helping organize raids, but later on even he (page 162) quotes a representative of the Irish weaver's guild effectively saying "we don't have your English problems of women working in the industry". In general it's a bit frustrating that the popular books on Luddism are all fairly uncritically pro-Luddist (though not surprising, I suppose). Merchant doesn't touch at all on the Luddist practice of going around to knitting-shops with women and "discharging them from working".

   ↩

4. Sometimes this is described as more troops that were sent to fight Napoleon (even by Merchant himself on page 89), but that isn't right.

   ↩

5. In quotes because it was not an official Luddist group (there were none), just people who were trying to stop the violence through lobbying and legislation.

   ↩

6. Otherwise why would any boss agree, instead of just waiting for the Luddites to do it themselves?

   ↩

6. I need an agent framework

7. I can have my agent write an agent framework!

8. Return to step 1

It's actually alarming how many friends and respected peers I've lost to this cycle already. Claude Code only got good maybe a few months ago, so this only recenlty started happening, so I assume they will emerge from the spiral eventually. I mean, I hope they will. We have no way of knowing.

Why we review

Anyway we know our symptom: the pipeline gets jammed up because of too much new code spewed into it at step 1. But what's the root cause of the clog? Why doesn't the pipeline go faster?

I said above that this isn't an article about AI. Clearly I'm failing at that so far, but let's bring it back to humans. It goes back to the annoyingly true observation I started with: every layer of review is 10x slower. As a society, we know this. Maybe you haven't seen it before now. But trust me: people who do org design for a living know that layers are expensive... and they still do it.

As companies grow, they all end up with more and more layers of collaboration, review, and management. Why? Because otherwise mistakes get made, and mistakes are increasingly expensive at scale. The average value added by a new feature eventually becomes lower than the average value lost through the new bugs it causes. So, lacking a way to make features produce more value (wouldn't that be nice!), we try to at least reduce the damage.

The more checks and controls we put in place, the slower we go, but the more monotonically the quality increases. And isn't that the basis of *continuous improvement?*

Well, sort of. Monotonically increasing quality is on the right track. But "more checks and controls" went off the rails. That's *only one way* to improve quality, and it's a fraught one.

"Quality Assurance" reduces quality

I wrote a few years ago about W. E. Deming and the "new" philosophy around quality that he popularized in Japanese auto manufacturing. (Eventually U.S. auto manufacturers more or less got the idea. So far the software industry hasn't.)

One of the effects he highlighted was the problem of a “QA” pass in a factory: build widgets, have an inspection/QA phase, reject widgets that fail QA. Of course, your inspectors probably miss some of the failures, so when in doubt, add a second QA phase after the first to catch the remaining ones, and so on.

In a simplistic mathematical model this seems to make sense. (For example, if every QA pass catches 90% of defects, then after two QA passes you’ve reduced the number of defects by 100x. How awesome is that?)

But in the reality of agentic humans, it’s not so simple. First of all, the incentives get weird. The second QA team basically serves to evaluate how well the first QA team is doing; if the first QA team keeps missing defects, fire them. Now, that second QA team has little incentive to produce that outcome for their friends. So maybe they don’t look too hard; after all, the first QA team missed the defect, it’s not unreasonable that we might miss it too.

Furthermore, the first QA team knows there is a second QA team to catch any defects; if I don’t work too hard today, surely the second team will pick up the slack. That's why they're there!

Also, the team making the widgets in the first place doesn’t check their work too carefully; that’s what the QA team is for! Why would I slow down the production of every widget by being careful, at a cost of say 20% more time, when there are only 10 defects in 100 and I can just eliminate them at the next step for only a 10% waste overhead? It only makes sense. Plus they'll fire me if I go 20% slower.

To say nothing of a whole engineering redesign to improve quality, that would be super expensive and we could be designing all new widgets instead.

Sound like any engineering departments you know?

Well, this isn’t the right time to rehash Deming, but suffice it to say, he was on to something. And his techniques worked. You get things like the famous Toyota Production System where they eliminated the QA phase entirely, but gave everybody an “oh crap, stop the line, I found a defect!” button.

Famously, US auto manufacturers tried to adopt the same system by installing the same “stop the line” buttons. Of course, nobody pushed those buttons. They were afraid of getting fired.

Trust

I suppose it worked, in the sense that it eventually succeeded in stopping the Luddist raids. But I can’t help but think that even a token gesture of compromise (say, requiring employers to make their wages public, or restricting the most cheap-and-nasty factory-made textile products) would have gone a long way towards calming things down. This almost actually happened! The 1812 Framework Knitters’ Bill, which had these provisions in it, passed the House of Commons but was shot down in the House of Lords.

Why did the government fail to even make a token attempt at compromise? Before the industrial revolution, I wonder if the workers and bosses of the English textiles industry were genuinely able to often just work out their problems together, so the government never really needed to do large-scale mediation. When that changed - when automation first made it possible for the bosses to durably “win” - government took a long time to realize, so there were some unpleasant decades of disempowered workers trying to bully factory-owners (via riots and death threats), and factory-owners trying to brutalize workers (via direct violence and automation).

Final thoughts

The Luddites weren’t anti-technology in general. Instead, they were against four or five specific machines that were automating away their skilled work. Contrary to many of the books I read, I think that’s actually fairly well understood today. But what surprised me was how truly decentralized and widespread the movement was. Every town with weavers faced the same incentives (the bosses to industrialize, and the workers to fight back) which created Luddism locally. And *nobody* was willing to report the Luddites: not for years, or for what would have been a fortune in cash, or in disagreement with their actions. They were only stopped by a truly fascist crackdown, with the army in the streets and the secret police pulling away random citizens for arrest and questioning.

I can see why modern “Luddites” - who are genuinely anti-technology - talk so much about the legacy of original Luddites. Luddism was a grassroots organization which notched up some real short-term wins, enjoyed near-total support among the public, and didn’t seem to be troubled by infighting at all[7]. If you’re an anti-AI campaigner, I bet all of that sounds great[8]. But I’m not convinced that the neo-Luddites really are the inheritors of Luddism. A load-bearing feature of Luddism is that it was *local*: it didn’t have manifestos, or leaders, or factions, or even much explicit ideology beyond the artisans’ immediate practical concerns. These were local men striking back against the local factories harming their local jobs. That simply isn’t the case with AI, where a datacenter in China can take my job in Australia.

If it isn’t time to start burning down datacenters, what can we do? Well, workers today are better off than the Luddites were, because we stand on their shoulders. Unlike the Luddites, today’s workers can all vote (and I suspect there will be no shortage of anti-AI candidates to vote for).

Why Luddism is not a good model for the anti-AI movement

There are many reasons why this doesn't map onto the current anti-AI movement. First, Luddism grew from a homogeneous group of high-status workers whose jobs almost vanished overnight, not a broad group of people whose jobs are getting slightly worse because of AI (like the gig-economy workers Merchant endlessly references). That meant that Luddites had *really specific* asks: higher wages for piecework, a phased introduction of specific textiles machinery, and so on. They were not generally demanding that the machines all be immediately destroyed[6].

Second, Luddism was very local. A pre-existing group of artisans in a particular town would gather in that town - either at work or an inn, say - and decide to petition or raid the businesses in that town that were harming their livelihoods. AI concerns are not like this. It isn't businesses in Chicago or Tokyo that are making decisions that imperil Chicago's or Tokyo's jobs, it's businesses in San Francisco. Unlike the Luddists, anti-AI activists can't naturally organize with people they already know to take direct action where they already live.

Third, Luddist *victory* could also be local. If you successfully lobby your local cloth business to not use a weaving machine, you have secured your job at that business for a while. But if you successfully lobby your town (or even your country!) to not build a datacenter, it doesn't meaningfully improve your local position, since your job can be as easily replaced by a datacenter on the other side of the planet.

A total failure of leadership

Reading through the history of the Luddites from a modern perspective, I was struck by the near-total absence of *good government*. The artisans were left to work out their grievances with their bosses more or less by themselves, with no formal channels for complaint or any attempt at mediation. When the government did intervene - in response to near-universal unrest in *half of the country* - they did this:

1. Make machine-breaking and oath-taking capital crimes
2. Dump thousands of soldiers more or less at random into the area, with no plan to guard factories or do anything beyond just hang around in case a revolution broke out
3. Empower a single magistrate to arrest and interrogate whoever he wanted in order to root out the conspiracy

The basis of the Japanese system that worked, and the missing part of the American system that didn't, is trust. Trust among individuals that your boss Really Truly Actually wants to know about every defect, and wants you to stop the line when you find one. Trust among managers that executives were serious about quality. Trust among executives that individuals, given a system that can work and has the right incentives, will produce quality work and spot their own defects, and push the stop button when they need to push it.

But, one more thing: trust that the system *actually does work*. So first you need a system that will work.

Fallibility

AI coders are fallible; they write bad code, often. In this way, they are just like human programmers.

Deming's approach to manufacturing didn't have any magic bullets. Alas, you can't just follow his ten-step process and immediately get higher quality engineering. The secret is, you have to get your engineers to engineer higher quality into the whole system, from top to bottom, repeatedly. Continuously.

Every time something goes wrong, you have to ask, "How did this happen?" and then do a whole post-mortem and the Five Whys (or however many Whys are in fashion nowadays) and fix the underlying Root Causes so that it doesn't happen again. "The coder did it wrong" is never a root cause, only a symptom. Why was it possible for the coder to get it wrong?

The job of a code reviewer isn't to review code. It's to figure out how to obsolete their code review comment, that whole class of comment, in all future cases, until you don't need their reviews at all anymore.

(Think of the people who first created "go fmt" and how many stupid code review comments about whitespace are gone forever. Now that's engineering.)

By the time your review catches a mistake, the mistake has already been made. The root cause happened already. You're too late.

Modularity

I wish I could tell you I had all the answers. Actually I don't have much. If I did, I'd be first in line for the Singularity because it sounds kind of awesome.

I think we're going to be stuck with these systems pipeline problems for a long time. Review pipelines — layers of QA — don't work. Instead, they make you slower while hiding root causes. Hiding causes makes them harder to fix.

But, the call of AI coding is strong. That first, fast step in the pipeline is *so fast!* It really does feel like having super powers. I want more super powers. What are we going to do about it?

Maybe we finally have a compelling enough excuse to fix the 20 years of problems hidden by code review culture, and replace it with a real culture of quality.

I think the optimists have half of the right idea. Reducing review stages, even to an uncomfortable degree, is going to be needed. But you can't just reduce review stages without something to replace them. That way lies the Ford Pinto or any recent Boeing aircraft.

The complete package, the table flip, was what Deming brought to manufacturing. You can't half-adopt a "total quality" system. You need to eliminate the reviews *and* obsolete them, in one step.

How? You can fully adopt the new system, in small bites. What if some components of your system can be built the new way? Imagine an old-school U.S. auto manufacturer buying parts from Japanese suppliers; wow, these parts are so well made! Now I can start removing QA steps elsewhere because I can just assume the parts are going to work, and my job of "assemble a bigger widget from the parts" has a ton of its complexity removed.

I like this view. I've always liked small beautiful things, that's my own bias. But, you can assemble big beautiful things from small beautiful things.

It's a lot easier to build those individual beautiful things in small teams that trust each other, that know what quality looks like *to them*. They deliver their things to customer teams who can clearly explain what quality looks like *to them*. And on we go. Quality starts bottom-up, and spreads.

I think small startups are going to do really well in this new world, probably better than ever. Startups already have fewer layers of review just because they have fewer people. Some startups will figure out how to produce high quality components quickly; others won't and will fail. Quality by natural selection?

Bigger companies are gonna have a harder time, because their slow review systems are baked in, and deleting them would cause complete chaos.

The Luddites also scared the hell out of the British government, who (encouraged by their over-eager spies) thought they might have a genuine revolution on their hands. While they didn't get many legal concessions at the time, the specter of Luddism must have loomed over the labor reform movement of the 1800s, which saw the first anti-child-labor laws and the beginnings of independent inspection of factories.

Finally, every book I read argued that the Luddism movement may have created the first idea of a "working class", by unifying many previously-independent groups of workers against a common enemy. Seen this way, the "political arm"[5] of Luddism can arguably claim partial credit for every labor victory since the 1800s (though the ringleaders were still hanged and the weavers did still lose their jobs).

The Luddist approach in a nutshell

We can now describe the "Luddist approach" to fighting technological change:

1. Find a few conspirators in your existing community who agree with your political project (but don't join a broader organization, since that leaves you vulnerable)
2. Make public anonymous demands in support of your specific goals, backed up by threats of violence, signed by a fictional character that's easy for other groups to appropriate
3. If your threats are ignored, attack the physical machines in the dead of night, destroying them and threatening (but not killing) any guards
4. Hope your example inspires many more people to independently do (1)-(3) themselves
5. Keep raiding, optionally escalating to assassination of some of the bosses, until you bait a totalitarian crackdown from the government
6. Eventually get arrested and executed, to great public dismay
7. Twenty years later, your example inspires the first national trade unions

Note that starting or joining a national movement is *not* the Luddist approach. Staying almost entirely isolated in small cells helped the Luddists avoid government spies and made them impossible to root out without enforcing a police state. Note also that you need a *lot* of public support for this to work: so that you get a lot of copycat groups without having to explicitly organize them, and so that your property destruction and murder is taken sympathetically instead of getting you immediately reported and arrested.

Because each group was so insular, any spies trying to infiltrate the movement would have been complete strangers to the community, and would thus have a very hard time gaining the trust of a group of men who had known each other for their whole lives. The spies that did exist were restricted to the occasional inter-group Luddist meetings, where people didn't all know each other so closely. But it's unclear how important those meetings were, since Luddist groups didn't need to coordinate to achieve their goals. According to Merchant, the spies spent much of their time embellishing tales of an imminent revolution to encourage their employers to keep the money flowing.

The crackdown

In the absence of reliable information, the British government was forced to use force. And they did, sending 12,000 troops[4] into the northern counties. This served mainly as an intimidation tactic, since there was no standing Luddite army to fight, and the soldiers spent most of their time marching back and forth or being abused by the townspeople.

More successful was the imposition of a full police state in Yorkshire, under the magistrate Joseph Radcliffe, who was empowered to randomly grab people off the street and interrogate them for days. That pressure eventually convinced a handful of people to give up their local Luddist organizers, who were tried and inevitably hanged. Their deaths (and the ensuing climate of fear) ended the high-water mark of Luddist activity. Even then, Luddist raids continued on and off for *six more years* before petering out.

Did the Luddites succeed?

This is a tricky question. In one sense the answer is obviously no: the movement was crushed, many of their leaders were executed, the textile industry continued to be automated, and today there are no longer thousands of jobs for skilled British weavers, knitters, spinners and dyers. The pro-automation side won.

However, they did achieve a number of short-lived victories. Their early threats often succeeded in preventing the building of a factory in a particular location, or in delaying the adoption of industrial machinery in a particular shop by years. In one case, hosiers that had been spooked by Luddite activity gave out pre-emptive bonuses to their workers to discourage them from smashing up their machines (which were indeed not smashed).

But, it's not just about company size. I think engineering teams at any company can get smaller, and have better defined interfaces between them.

Maybe you could have multiple teams inside a company competing to deliver the same component. Each one is just a few people and a few coding bots. Try it 100 ways and see who comes up with the best one. Again, quality by evolution. Code is cheap but good ideas are not. But now you can try out new ideas faster than ever.

Maybe we'll see a new optimal point on the monoliths-microservices continuum. Microservices got a bad name because they were too micro; in the original terminology, a "micro" service was exactly the right size for a "two pizza team" to build and operate on their own. With AI, maybe it's one pizza and some tokens.

What's fun is you can also use this new, faster coding to experiment with different module *boundaries* faster. *Features* are still hard for lots of reasons, but refactoring and automated integration testing are things the AIs excel at. Try splitting out a module you were afraid to split out before. Maybe it'll add some lines of code. But suddenly lines of code are cheap, compared to the coordination overhead of a bigger team maintaining both parts.

Every team has some monoliths that are a little too big, and too many layers of reviews. Maybe we won't get all the way to Singularity. But, we can engineer a much better world. Our problems are solvable.

It just takes trust.

Related

Highlights on "quality," and Deming's work as it applies to software development (2016)

Systems design 3: LLMs and the semantic revolution (2025)

Unrelated

SimSWE part 2: The perils of multitasking (2017)

Luddites and AI datacenters

SEANGOEDECKE.COM RSS FEED · 22 APR 2026 · SOURCE

Is it time to start burning down datacenters?

Some people think so. An Indianapolis city council member had his house recently shot up for supporting datacenters, and Sam Altman's home was firebombed (and then shot) shortly afterwards. People from all sides of the argument are sounding the alarm about imminent violence.

The obvious historical comparison is Luddism, the 19th-century phenomenon where English weavers and knitters destroyed the machines that were automating their work, and (in some cases) killed the machines' owners. Anti-AI people are reclaiming the term to describe themselves, and many of the leading lights of the anti-AI movement (like Brian Merchant or Gavin Mueller) have written books arguing more or less that the Luddites were right, and we ought to follow their example in order to resist AI automation[1].

Like many people, I have heard a lot about Luddism and Luddites, but only in the context of it being a general term for someone who is anti-technology. I was interested in learning more about the actual historical movement: what kind of people participated, what it was, and what it accomplished. I read Merchant's and Mueller's books, plus others[2], to try and figure all of this out. Who were the actual, historical Luddites? What can we learn from them about burning down datacenters?

Who were the Luddites?

The Luddites were a decentralized movement of artisans in the 1810s who engaged in violent protest - smashing machines, threatening violence, and ultimately killing people - over the fact that their jobs were being automated away. They were not rich, but they were certainly not unskilled labor: these were people who had apprenticed for seven years. They were mostly working from home, producing cloth from raw material given to them by their employer, often with tools rented from that same employer. They were working short weeks (three days, per William Gardiner) at their own discretion.

In the early 1800s, their skilled labor was becoming unnecessary. With the help of expensive machines, unskilled labor could now produce lower-quality cloth, so employers were beginning to pass over these artisans in favor of cheaper employees: children, unapprenticed workers, and women[3]. Combined with the bad economic position of England at the time (at war with France, and thus deliberately cutting off much European trade), times were beginning to be very tough indeed. Starvation was a real threat.

What did they do?

Cloth artisans were groups of capable men who were used to getting their own way, knew each other very well, and were broadly respected in their communities. It was thus a natural response for them to organize into what was effectively a militant union. The Luddites would send anonymous threatening letters to their old (or current) bosses, warning them to stop using their machines. If they didn't comply, they would raid the workshop or factory, smashing the machines up.

They typically did not harm people, though they certainly delivered threats of bodily harm or even murder, and the raids were violent enough (e.g. shooting through windows) to have risked accidental deaths. In at least two instances where a factory owner was seen as unusually cruel, the Luddites did attempt assassinations: one unsuccessful, and one successful one that eventually prompted a crackdown that ended the movement for good.

Luddism was fully decentralized. Different communities could and did decide to engage in machine-raiding independently, particularly when news spread of the tactic succeeding. Although each community had its own influential men, there was never a single "leader of Luddism". King Ludd himself was a folk-tale figure. This made it an absolute nightmare for the British government to try and suppress them: putting down one Luddist group did nothing to prevent other groups from continuing to operate.

All the king's spies

I was surprised by how *difficult* it was for the government to get a hold of any of the local Luddist ringleaders. The government was willing to offer huge rewards to informers: at one point up to 40x the yearly wage. However, there were no takers for several years. Armies of spies were recruited and tasked with infiltrating Luddist groups, with absolutely no success.

Why was it so hard? Firstly, because the working class was so overwhelmingly pro-Luddist. People universally blamed the economic situation on the government and the factory owners (rightfully so, since the government had chosen to go to war and the factory owners had chosen to embrace automation). Secondly, the communities in question were so insular and tightly-knit that informers would have to rat on their friends and relatives. The handful of people who did eventually inform lived out the rest of their lives as pariahs.